Projet - Algèbre Linéaire Numérique

POLYTECH - LILLE

2023

IRIS DEPAGNEUX

VINCENT LE MEUR

IS3

Professeurs référents:

François Boulier

François Lemaire

Sommaire

Sommaire **1**
Introduction : **2**
Objectifs 2
Résolution 2
I- Utiliser les fichiers **3**
II- Réalisation des functions **6**
I- Programmer scipy.linalg.eigh 6
II- Programmer scipy.linalg.svd 8
Conclusion **9**

Introduction :

Le projet consiste à reprogrammer plusieurs fonctions incluses dans le package Python : scipy.linalg et à les appliquer sur un exemple (Ici sur les Iris).

Objectifs

1. Reprogrammer la fonction scipy.linalg.eigh
2. Reprogrammer la fonction scipy.linalg.svd

Résolution

Lors de ce projet nous avons programmé deux fonctions issues du package scipy.linalg: eigh et svd. Pour réussir cela nous avons aussi dû programmer les fonctions hessenberg, QR et solve qui sont utilisés dans nos fonctions.

Pour vérifier nos fonctions nous avons utilisé plusieurs tests récupérés dans le cours et les TD qui nous permettent ainsi d'observer les succès et échecs de nos fonctions comparées à celles du package.

Pour ce projet nous avons décidé de prendre comme exemple principal l'acp iris présent dans le fichier iris.py.

Lors de ce projet on s'est réparti le travail entre nous de façon à ce que chacun s'occupe majoritairement d'une fonction sur les deux.

On à rencontré plusieurs difficultés durant le projet: notamment sur la programmation du code pour la svd.

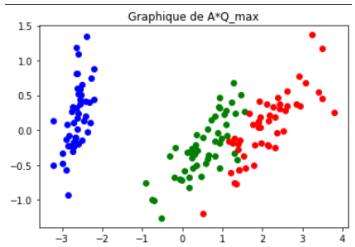
I- Utiliser les fichiers

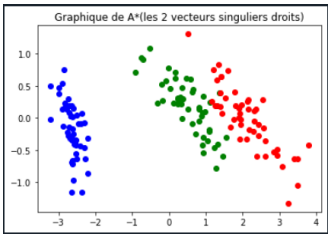
- Le code principal avec les fonctions non changées se situe dans le fichier *acp_iris.py*. Lorsqu'on l'exécute les valeurs propres, vecteurs propres, U , Sigma et Vt sont affichés sur la sortie du terminal, on observe aussi les graphiques que l'on doit obtenir.

```
In [14]: runcell(0, '/home/v/l/vlemeur/IS3/S6/ALN/projet-aln-2023/acp_iris.py')
--------------------Méthode nla.eigh-------------------------------
Valeurs propres:
[  1.17625701   3.90002077  12.03143072 209.83375816]
Vecteurs propres:
[[ 0.31725455 -0.58099728  0.65653988  0.36158968]
 [-0.32409435  0.59641809  0.72971237 -0.08226889]
 [-0.47971899  0.07252408 -0.1757674   0.85657211]
 [ 0.75112056  0.54906091 -0.07470647  0.35884393]]

Figures now render in the Plots pane by default. To make them also appear inline in the Console,
uncheck "Mute Inline Plotting" under the Plots pane options menu.

 --------------------Méthode nla.svd--------------------------------
Sigma:
[25.08757569  5.47771923  0.08556067  0.02827037]
Vt:
[[ 0.36170507 -0.07691353  0.85302495  0.36824201]
 [ 0.81284709  0.47801971 -0.18098766 -0.27932102]
 [ 0.00842191  0.36125867 -0.34495811  0.86627083]
 [ 0.45648453 -0.79691508 -0.34726905  0.18961129]]
```





- Dans le fichier *eigh.py* se trouve le code pour remplacer la fonction scipy.linalg.eigh, cette fonction renvoie les valeurs propres ainsi que les vecteurs propres de la matrice donnée.
- De même, dans le fichier *bidiagonale.py* se trouve le code pour remplacer la fonction scipy.linalg.svd. Cette fonction renvoie les matrices U et Vt ainsi que les coefficients sigma de la matrice donné .

Pour vérifier ces fonctions nous avons créé deux fichiers: test_eigh.py et test_bidiago.py qui vérifient respectivement les fonctions eigh et svd. Lorsqu'ils sont exécutés, affichent le succès si la fonction programmée renvoie la même chose que celle du package et l'échec sinon.

```
In [16]: runcell(0, '/home/v/l/vlem
Exécution de test_eigh_1 : succès
Exécution de test_eigh_2 : succès
Nombre de tests  : 2
Nombre de succès : 2
Nombre d'échecs  : 0
```

Pour celui de eigh on observe un premier succès avec les valeurs propres de même pour les vecteurs propres, cependant on doit utiliser la valeur absolue pour l'obtenir.

```
Exécution de test_eigh_1 : échec
Exécution de test_eigh_2 : succès
Exécution de test_eigh_3 : échec
Nombre de tests  : 3
Nombre de succès : 1
Nombre d'échecs  : 2
```

Pour la fonction svd on obtient un succès sur le sigma , cependant on obtient des échecs pour le U et le Vt.

```
Exécution de test_eigh_3xbar
[[ 0.39144351 -0.87964546  0.27017743]
 [-0.81442356 -0.46785084 -0.34326352]
 [-0.60578253  0.12115651  1.27214332]]
x
[[-0.39144351  0.87964546 -0.27017743]
 [-0.81442356 -0.46785084 -0.34326352]
 [ 0.42835294 -0.08567059 -0.89954117]]
```

En regardant plus précisément on obtient les mêmes valeurs en valeur absolue pour vt sauf pour la dernière ligne.

```
Exécution de test_eigh_1xbar
[[-8.27758420e-17  3.47406172e-01  7.38901630e-01]
 [-2.06939605e-16 -8.13610668e-01 -6.85882446e-02]
 [ 2.89715447e-16  4.66204496e-01 -6.70313385e-01]]
x
[[-0.73890163  0.34740617  0.57735027]
 [ 0.06858824 -0.81361067  0.57735027]
 [ 0.67031339  0.4662045   0.57735027]]
```

Pour U on obtient à peu près les bonnes valeurs sur la bidiagonale ou la première et dernière colonnes sont inversées écrites en écriture scientifique mais pas sur le reste, ce qui nous importe peu car on à besoin que de la bidiagonal.

- On à un autre exemple dans le fichier *test.py* qui utilise une matrice vu en cours. On applique alors nos fonctions sur l'exemple et on les compare avec celles du package.

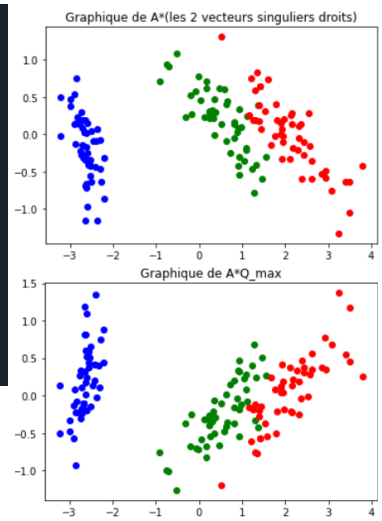
```
In [21]: runcell(0, '/home/v/l/vlemeur/IS3/S6/ALN/projet-aln-2023/tes
--------------------Méthode nla.svd--------------------------------
Les valeurs propres à trouver:
[8.8817842e-16 2.0000000e+00 1.2500000e+01]
Vecteurs propres à trouver :
[[-1.79056769e-16  8.00000000e-01 -6.00000000e-01]
 [-8.00000000e-01 -3.60000000e-01 -4.80000000e-01]
 [ 6.00000000e-01 -4.80000000e-01 -6.40000000e-01]]
Valeurs propres du test:
[-3.55271368e-16  2.00000000e+00  1.25000000e+01]
Vecteurs propres du test :
[[ 1.59872116e-16 -8.00000000e-01  6.00000000e-01]
 [ 8.00000000e-01  3.60000000e-01  4.80000000e-01]
 [-6.00000000e-01  4.80000000e-01  6.40000000e-01]]
```

```
---------------Programme de la Méthode nla.svd------------------------
Le U à trouver:
[[-0.5         0.5        -0.68671386]
 [ 0.5        -0.5        -0.68671386]
 [ 0.5         0.5        -0.16859442]
 [-0.5        -0.5        -0.16859442]]
Le Sigma à trouver:
[5.00000000e+00 2.00000000e+00 6.58517064e-16]
Le Vt à trouver:
[[ 0.6   0.48  0.64]
 [ 0.8  -0.36 -0.48]
 [ 0.   -0.8   0.6 ]]
Le U du test:
[[ 0.70710678  0.5        -0.5       ]
 [ 0.70710678 -0.5         0.5       ]
 [ 0.70710678  0.5         0.5       ]
 [ 0.70710678 -0.5        -0.5       ]]
Le Sigma du test:
[5. 2. 0.]
Le Vt du test:
[[ 0.6   0.48  0.64]
 [ 0.8  -0.36 -0.48]
 [ 0.    0.    0.  ]]
```

On obtient les mêmes observations qu'avec les tests.

- Pour terminer dans le fichier *final.py* on observe le code acp_iris ou l'on utilise cette fois nos fonctions. Comme pour acp_iris.py on obtient sur la sortie du terminal les variables à obtenir et les graphiques.

```
In [23]: runcell(0, '/home/v/l/vlemeur/IS3/S6/ALN/projet-aln-2023/final.py')
---------------Programme de la Méthode nla.eigh-----------------------
Valeurs propres:
[  1.17625701   3.90002077  12.03143072 209.83375816]
Vecteurs propres:
[[ 0.31725455 -0.58099728  0.65653988  0.36158968]
 [-0.32409435  0.59641809  0.72971237 -0.08226889]
 [-0.47971899  0.07252408 -0.1757674   0.85657211]
 [ 0.75112056  0.54906091 -0.07470647  0.35884393]]
---------------Programme de la Méthode nla.svd------------------------
Sigma:
[25.08757569  5.47771923  0.08556067  0.02827037]
Vt:
[[ 0.36170507 -0.07691353  0.85302495  0.36824201]
 [-0.81284709 -0.47801971  0.18098766  0.27932102]
 [-0.00842191 -0.36125867  0.34495811 -0.86627083]
 [ 0.45648453 -0.79691508 -0.34726905  0.18961129]]
```



En comparant les 2 résultats on remarque que l'on obtient les mêmes valeurs et les mêmes graphes.

En conclusion, grâce aux exemples et aux tests, on observe que la fonction eigh que l'on a programmée remplace parfaitement la fonction eigh du package scipy.linalg. En observant seulement l'exemple iris qui nous intéresse, on observe que l'on obtient les mêmes valeurs pour sigma et vt, mais avec les tests on observe quelques différences. Cela nous permet de valider à moitié notre fonction programmée svd pour remplacer svd du package.

II- Réalisation des functions

I- Programmer scipy.linalg.eigh

La fonction permettant de remplacer la fonction eigh du package scipy.linalg se trouve dans le fichier eigh.py, elle se nomme *vap_vep* (l.116) et utilise 3 fonctions différentes:

```
def vap_vep(A):
    # Calcul des valeurs propres et vecteurs propres d'une matrice
    eigenvectors = []
    eigenvalue = []

    hessenberg_matrix = hessenberg(A) #remplace la fonction nla.hessenberg

    eigenvalues = qr_algorithm(hessenberg_matrix) #remplace la fonction nla.qr
    eigenvalues = np.flip(eigenvalues)

    for eigenvalue in eigenvalues:
        eigenvector2 = inverse_power_iteration(A, eigenvalue)
        eigenvectors.append(eigenvector2)

    matrix = np.vstack(eigenvectors)
    matrix = np.transpose(matrix)

    return eigenvalues, matrix
```

- Dans un premier temps on transforme la matrice donnée en matrice de hessenberg, pour cela une fonction du package scipy.linalg est disponible: scipy.linalg.hessenberg. Cependant nous l'avons aussi reprogrammé sous le nom de *hessenberg* (l.6), elle effectue une série d'opérations pour transformer la matrice d'entrée en sa forme de Hessenberg correspondante, où tous les éléments en dessous de la première sous-diagonale sont nuls.

```
def hessenberg(matrix):
    # Calcul de la forme de Hessenberg d'une matrice
    n = matrix.shape[0]
    hessenberg_matrix = matrix.copy().astype(float)
    for k in range(1, n - 1):
        x = hessenberg_matrix[k:, k-1]
        v = x.copy()
        v[0] += np.sign(v[0]) * np.sqrt(np.dot(v, v))
        v = v / nla.norm(v)
        hessenberg_matrix[k:, k-1:] -= 2 * np.outer(v, np.dot(v, hessenberg_matrix[k:, k-1:]))
        hessenberg_matrix[:, k:] -= 2 * np.outer(np.dot(hessenberg_matrix[:, k:], v), v)
    return hessenberg_matrix
```

- Ensuite pour trouver les valeurs propres on utilise l'algorithme QR, une fonction est aussi disponible: scipy.linalg.qr que l'on a reprogrammé et nommé: *qr_algorithm* (l.35), Elle calcule les matrices Q et R telles que Q est une matrice orthogonale et R est une matrice triangulaire supérieure. Dans cette fonction on utilise une autre fonction que l'on à programmé pour pouvoir faire une décomposition qr: *qr_decomposition* (l.19).

```python
def qr_algorithm(matrix, max_iterations=1000, tolerance=1e-10):
    # Algorithme QR pour calculer les valeurs propres d'une matrice
    n = matrix.shape[0]
    eigenvalues = np.zeros(n)
    for i in range(max_iterations):
        q, r = qr_decomposition(matrix)
        matrix = np.dot(r, q)
        eigenvalues_new = np.diag(matrix)
        eigenvalues = eigenvalues_new
    return eigenvalues
```

```python
def qr_decomposition(matrix):
    # Décomposition QR d'une matrice
    m, n = matrix.shape
    q = np.zeros((m, n))
    r = np.zeros((n, n))

    for j in range(n):
        v = matrix[:, j].copy()
        for i in range(j):
            r[i, j] = np.dot(q[:, i], matrix[:, j])
            v -= r[i, j] * q[:, i]
        r[j, j] = nla.norm(v)
        q[:, j] = v / r[j, j]

    return q, r
```

- Puis, pour trouver les vecteurs propres on utilise la méthode de la puissance inverse que l'on traduit sous le nom de *inverse_power_iteration* (l.105). Pour cette méthode on doit résoudre un système linéaire avec grâce aux valeurs propres en utilisant la fonction *custom_solve* (l.70) au lieu d'utiliser la fonction scipy.linalg.solve.

```python
def inverse_power_iteration(A, eigenvalue, tolerance=1e-10, max_iterations=1000):
    # Méthode de la puissance inverse pour trouver le vecteur propre correspondant à une valeur propre donnée
    n = A.shape[0]
    x = np.ones(n)
    x = x / nla.norm(x)
    for i in range(max_iterations):
        x = custom_solve(A - eigenvalue * np.eye(n), eigenvalue, x)#remplace nla.solve
        x = x / nla.norm(x, 2)
    return x
```

Enfin la fonction vap_vep retourne les valeurs propres ainsi que les vecteurs propres de la matrice donnée.

II- Programmer scipy.linalg.svd

La fonction qui permet de remplacer la fonction scpicy.svd se trouve dans le fichier *bidiagonale.py* et se nomme *decom_val_sing* (l.25).

Pour la programmer on utilise un autre fonction qui calcule à partir d'un vecteur v, une matrice de réflexions de Householder. Cette fonction se nomme dans le fichier *reflecteur* (l.16).

La fonction principale est séparée en plusieurs parties.:

```python
def decom_val_sing(A):
    m, n = A.shape
    B = np.copy(A)
    x = B[0:m, 0]
    v = np.copy(x)
    v[0] = v[0] + np.sign(v[0]) * nla.norm(x, 2)
    v = (1 / nla.norm(v, 2)) * v
    VL = [v]
    Q = reflecteur(m, v)
    B = np.dot(Q, B)
    x = B[0, 1:n]
    v = np.copy(x)
    v[0] = v[0] + np.sign(v[0]) * nla.norm(x, 2)
    v = (1 / nla.norm(v, 2) )* v
    VR = [v]
    Q = reflecteur(n, v)
    B = np.dot(B, Q)
    x = B[1:m, 1]
    v = np.copy(x)
    v[0] = v[0] + np.sign(v[0]) * nla.norm(x, 2)
    v = (1 / nla.norm(v, 2) )* v
    VL.append(v)
    Q = reflecteur(m, v)
    B = np.dot(Q, B)
    B=B[0:n,0:n]
    H=np.zeros([2*n,2*n],dtype=np.float64)
    H[0:n,n:2*n]=np.transpose(B)
    H[n:2*n,0:n]=B
    P=np.zeros([2*n,2*n],dtype=np.float64)
    for i in range(0,n):
        P[i,2*i]=1
        P[n+i,2*i+1]=1
```

```python
T=np.dot(np.transpose(P),np.dot(H,P))
d=np.zeros(2*n,dtype=np.float64)
e=np.array([T[i+1,i]for i in range (0,2*n-1)],dtype=np.float64)
eigvals, eigvecs=nla.eigh_tridiagonal(d, e)
Lambda=eigvals[n:2*n]
Q=eigvecs[:,n:2*n]
Y=np.sqrt(2)*np.dot(P,Q)
newVt=np.transpose(Y[0:n,:])
newVt = newVt[::-1]
newU=np.zeros([m,n],dtype=np.float64)
newU[0:n,:]=Y[n:2*n,:]
newSigma=np.array(np.diag(Lambda),dtype=np.float64)
newSigma= np.flip(newSigma)
for i in range (len(VL)-1,-1,-1):
    Q=reflecteur(m,VL[i])
    newU=np.dot(Q,newU)



for i in range(len(VR)-1,-1,-1):
    Q=reflecteur(n, VR[i])
    newVt=np.dot(newVt,Q)

Sig=newSigma.diagonal()

return newU ,Sig,newVt
```

- Au début nous avons construit une matrice bidiagonale B à partir de A. Cependant la matrice donnée par ce code ne donne pas exactement une matrice bidiagonale car elle ne remplace pas les termes qui ne sont pas sur la bidiagonale par 0, heureusement cela n'aura pas d'impact sur la suite.
- L'étape suivante a pour but de trouver sigma à l'aide du calcul de la matrice de permutation P et une matrice de bloc H venant de B. Grâce à cela on obtient les valeurs propres et leurs vecteurs propres associés. Ce qui nous permet de calculer newsigmas.
- Puis on calcul new U et new Vt. Cela nous a posé des problèmes car nous avons eu du mal à trouver les bonnes bornes pour les boucles de création de newVt et newU.

Enfin la fonction retourne les matrices U et VT ainsi que sigma.

Conclusion

Pour conclure, on à réussi malgré les problèmes rencontrés à programmer deux fonctions *vap_vep* et *decom_val_sing* qui permettent de remplacer les fonctions du package scipy.linalg eigh et svd en mettant en pratique ce que l'on à vu en td et cours: la matrice d'Hessenberg, l'algorithme et la factorisation QR, la méthode la puissance inverse....